Color Block Crush Solver

PlaySimple Hackathon

Sunil
Roll No: 22PD36

September 8, 2026

Contents

1 Introduction

A level is a grid of coloured polyomino blocks. Each block slides in a straight line until it hits a wall, another block, or a matching-colour gate cut into the border, and it never stops halfway on its own; some blocks are pinned to one axis, and some are held in place by ice until a fixed number of other blocks have already left the board. The tricky part is that letting one block exit can immediately open a gap for another block, mid-move, and that chain reaction has to resolve in exactly the right order or a level that is actually solvable gets reported as stuck.

`solve.py` reads one of these levels and prints a status line, a move count, and then the moves themselves, one block-and-destination pair per line. Nothing gets printed until it has been replayed through the same simulator that ran the search, so a bug in the search code cannot quietly hand back a move list that does not actually work. There are three solvers behind the same CLI flag `-solver`: `complete`, `fast`, and `auto`, which is the default because it is the one you actually want most of the time.

2 What the Solver Actually Guarantees

Three things matter, roughly in this order, and they are not equally hard to get right.

2.1 Correctness comes first

Every printed solution is replayed against the real board rules before it reaches stdout. A wrong move list is worse than a slow one, so this check runs unconditionally, not just in tests.

2.2 An UNSOLVABLE verdict has to be trustworthy

Nothing distinguishes "the solver gave up" from "there is truly no solution" once the process exits, so the only way any of the three engines report `UNSOLVABLE` is by exhausting a search whose heuristic can return infinity, and it only does that for a block that provably cannot reach any exit on a board stripped down to just its walls. Weighted or greedy search phases never touch this verdict; they only reorder the queue, they do not remove states from it.

2.3 Speed and move count, in that order

On the five levels supplied with the assignment, the slowest single run takes about twenty seconds (`complete` on test4, the busiest board), comfortably inside a sixty-second budget. Move quality is where the trade-off actually lives, and `auto` closes it: it matches `complete`'s shortest answer on all five levels, 2, 10, 24, 49 and 43 moves respectively.

3 How a Board State Is Stored

A search state is a pair: one anchor cell per block (or a sentinel value once that block has exited) plus a bitmask marking which blocks are already out. Two lookup tables built once, when the level is parsed, do almost all of the remaining work. `masks[block][anchor]` gives the exact set of board cells a block occupies at a given anchor as a single integer, so checking whether two placements collide is one bitwise `AND`. `steps[block][direction][anchor]` gives the next anchor one cell over in that direction, or -1 at a wall, which turns sliding into a plain table walk instead of a loop over coordinates.

The trickier problem is blocks that are identical in every way that matters: same shape, same colour, same axis restriction, same ice threshold. Test 4 has four such `DB` 1×1 blocks and Test 5 has eight `Y` 1×1 blocks, and treating those as distinguishable would multiply the reachable state space by 4! and 8! for no reason at all, since swapping two identical blocks produces a board that looks and plays exactly the same. The fix is a canonical key: identical blocks are grouped together when the level is loaded, and the key used to detect a repeated state sorts each group's anchors before hashing, so "block A at cell 12, block B at cell 40" and "block A at cell 40, block B at cell 12" collapse to the same key. Keys are stored as raw `bytes` rather than tuples, partly for hashing speed and partly because Test 4's exact search alone ends up storing just over 300,000 of them, and a search is capped at 2.5 million stored states so a runaway level times out instead of getting killed by the operating system for using too much memory.

4 Move Generation and the Exit Cascade

Generating moves for a block means sliding it one cell at a time until the next cell would collide with a wall or another block, and every intermediate stop along that slide is a legal move on its own, not only the farthest one it can reach. That is a deliberate choice: at least one of the given test levels only has a solution that uses a partial slide, so treating only the farthest stop as legal would make some of these levels look unsolvable when they are not, and given how much section 2 leans on `UNSOLVABLE` being sound, that is not a corner worth cutting.

Ice-locked blocks are simply skipped by the move generator until enough other blocks have exited, and blocks restricted to one axis only ever get slides in that axis's two directions instead of four. The interesting part happens after a move lands: if the new placement lines up with that block's matching-colour gate, the engine does not just remove the block and move on. It runs a small fixpoint loop, repeatedly checking every block that is sitting in front of an exit for a clear path and an ice threshold that is now satisfied, releasing any that qualify, and looping again until nothing more releases in that same transition. Test 5 needs this: one block sits on its own gate from the very start of the level with an ice threshold of one, and the block whose exit actually satisfies that threshold leaves in the very same move. Checking exits only once before and once after the move, rather than inside that loop, would strand the ice-locked block and the level would come back as unsolvable when it is not.

5 Heuristics

Two different heuristics are computed here, and they are kept deliberately separate rather than blended into one number, because they answer two different questions.

5.1 The admissible one: how far, at minimum

For every block, a backward breadth-first search runs once from that block's own exit cells, over a board that contains only walls, no other blocks. That gives a table of the minimum number of slides the block would need if nothing else were in its way. Summing this over every block still on the board never overstates the true remaining cost, because a real move can only ever reduce one block's distance by exactly one slide, so this sum is admissible, which is exactly what `complete`'s shortest-path guarantee and the `UNSOLVABLE` check both depend on.

5.2 The greedy one: what's actually in the way

The admissible number above has no idea that two blocks might be blocking each other, so a second, deliberately non-admissible heuristic looks at the straight-line corridor each pending block would sweep on its way to its nearest exit and counts how many other blocks currently sit inside it, then goes one level deeper and checks whether those blockers are themselves blocked (capped at four, to keep this cheap). A flat penalty is added for every block still frozen behind an unmet ice threshold. This is what actually makes the busy levels tractable: on Test 4, the admissible distance alone expands roughly a quarter of a million states before finding any answer, while adding this guide term to a weighted search cuts that down to under six thousand.

6 Search Algorithms

6.1 Why plain BFS is not the answer here

Before any heuristic gets involved, the repository keeps a completely uninformed breadth-first search around, `search/bfs.py`, mostly as a control. It is the textbook version: a FIFO queue, a real closed set (there is no reopening question at all here, since every move costs exactly one step and a state marked closed is genuinely never worth revisiting), and it expands states purely in the order they were discovered, with no notion of which ones look promising. On Test 1, a two-move level, that is more than good enough and it returns instantly. On every other given level it runs out of its time budget without finding anything, because it has no way to tell a move that heads toward an exit from one that wanders off in the wrong direction, and the reachable state space on a busier board is simply too large to search blind. That gap between test1 and the rest is the actual argument for building a heuristic at all; it would be easy to add one and just assume it helps.

6.2 Weighted A*, run three different ways

`complete`, `fast`, and `auto` are not three separate algorithms. They are one algorithm, weighted A*, run at three different weight settings on the class `BestFirstSearch` in `search/astar.py`. What it computes on every pop from the priority queue is $f(n) = g(n) + w \cdot h_{\text{adm}}(n) + w_g \cdot h_{\text{guide}}(n)$, where g is the number of moves taken so far, h_{adm} is the admissible per-block distance from Section 5, and h_{guide} is the non-admissible corridor term, added only when a solver asks for it.

Its one real departure from a textbook A* is that it never keeps a closed set. Instead of marking a state permanently done once it is expanded, the engine remembers only the shallowest depth at which each state has been seen so far, and if a cheaper path to an already-expanded state turns up later, that state is simply reopened and pushed back onto the queue rather than ignored. For a consistent heuristic this costs nothing extra, because no state is ever reached more cheaply after it has already been expanded in the first place; the payoff shows up once the search is run in multiple phases, since an anytime scheme built this way does not need the separate bookkeeping list for reopened nodes that this kind of search normally carries.

`complete` runs that search once, at weight 1, on the admissible distance alone; that is plain A*, and whatever it returns first is provably shortest, while an empty queue with nothing found means the level really is unsolvable. `fast` runs the identical search once at weight 3, with the greedy guide term switched on, which is closer to greedy best-first search than to A* proper: it gives up roughly ten extra moves on the busier levels in exchange for finishing in a few seconds instead of twenty, and because it still shares the same move generator and exit rules as `complete`, an `UNSOLVABLE` answer from `fast` is just as trustworthy, only its move-count optimality is gone.

Solver	Algorithm	w	w_g	Optimality claim
`complete`	Weighted A*, single pass	1.0	0	Shortest path, proven
`fast`	Weighted A*, single pass	3.0	3.0	None
`auto`	Weighted A*, four passes	$3 \to 1$	$3 \to 0$	Shortest path, proven

Table 1: The same search class, `BestFirstSearch`, run at three different settings.

`auto`, the default, does not choose between the two. It runs one search through a shrinking weight schedule, $(3, 3) \to (2, 2) \to (1.5, 1.5) \to (1, 0)$, which is the same idea behind the anytime search technique usually called ARA*: start greedy, find something quickly, then lower the weight and keep refining. Whenever it lands on a goal partway through that schedule, that answer becomes the current best guess, tightens a bound used to prune the queue, and the search carries on into the next, less greedy phase on the same open list, rather than throwing everything away and restarting `complete` from nothing. If the queue ever runs dry with a best guess already on hand, that guess is certified optimal at any weight, because the bound used for pruning is always the admissible term, never the greedy one, so nothing on a genuinely shorter path could have been dropped along the way.

7 Results

These numbers are from a single run of each solver against each of the five levels supplied with the assignment, on one core of an ordinary Linux laptop. Wall-clock time moves around by roughly twenty percent between runs depending on what else the machine is doing; move counts and the number of states expanded do not change at all, since the search itself is deterministic.

	complete			**fast**			**auto**	
Level	Moves	Expanded	Time (s)	Moves	Expanded	Time (s)	Moves	Time (s)
test1	2	2	0.00	2	2	0.00	2	0.00
test2	10	10	0.00	11	11	0.01	10	0.01
test3	24	64,301	4.02	32	327	0.11	24	5.42
test4	49	259,396	16.61	58	5,807	3.48	49	20.05
test5	43	16,312	0.62	53	11,439	2.61	43	5.02

Table 2: All three solvers on the five given levels. `complete` and `auto` always agree on move count; `expanded` is the number of states popped off the queue before the answer was found.

`fast` trades roughly nine to ten moves for somewhere between six and thirty times fewer expanded states on the two busy levels, test3 and test4, which is the whole reason it exists. `auto` does not make that trade at all; it matches `complete`'s move count on every single one of these five levels, and the price for that is paid in wall-clock time on the levels `complete` was already fast on, test5 going from 0.62 seconds under `complete` alone to 5.02 seconds under `auto`, since `auto` has no way to know in advance that the later, slower phases of its schedule will not be needed. That is a real cost, not a rounding error, and it is worth stating plainly rather than only showing the levels where the trade looks free.

8 Summary

All three solvers run on the same move generator and the same exit-cascade simulator, so correctness never changes between them, only how greedy the search is allowed to be. That is why `auto` works as the default: it is not a fourth algorithm, it is the same weighted A* run longer with the weight turned down step by step. The one trade this project makes is moves against time, never moves against correctness, and every number in Section 7 came from actually running the solver, not from a guess.